Blatt 07: C++

Carsten Gips, BC George (HSBI)

1 Zusammenfassung

Ziel dieses Aufgabenblattes ist die praktische Anwendung Ihrer C++-Kenntnisse, insbesondere werden Sie Pointer, Referenzen und Klassen anwenden und vertiefen. Als Anwendungsbeispiel werden Sie bestimmte in der C++-Welt wohlbekannte Smart-Pointer modellieren sowie einen einfachen Ringpuffer programmieren. Sie lernen mit dem *Reference Counting* nebenbei eine verbreitete Technik der *Garbage Collection* kennen.

2 Methodik

Sie werden auf diesem Blatt vier einfache Klassen in C++ implementieren.

Es empfiehlt sich, zunächst die Beispiele gründlich zu analysieren, um die gewünschte Funktionsweise der einzelnen Klassen vorab präzise zu verstehen. Sie werden zu einigen Dingen in der C++-Literatur recherchieren müssen.

Implementieren Sie immer eine Klasse vollständig und testen Sie Ihren Code sowohl mit den vorgegebenen Beispielen als auch mit eigenen Beispielen, bevor Sie sich an die nächste Aufgabe/Klasse setzen.

3 Speicherverwaltung in C/C++

C und C++ erlauben als hardwarenahe Programmiersprachen den direkten Umgang mit dem Programmspeicher (Heap). Ein Programm kann dynamisch zu jeder Zeit weiteren Speicher anfordern und so beispielsweise mitwachsende Datenstrukturen realisieren.

Da der Heap-Speicher endlich ist, muss man nicht mehr benötigten Speicher auch wieder freigeben. Anderenfalls ist irgendwann der komplette Heap belegt und das Programm kann nicht mehr ordnungsgemäß arbeiten. Für die Freigabe ist man als Programmierer:in selbst zuständig.

3.1 Beispiel für eine Tokenizer-Funktion

Im folgenden Programmschnipsel soll eine Funktion `next_token()` das nächste Token berechnen. So eine Funktion findet sich typischerweise im Lexer. Für die Rückgabe des Tokens hat man in C++ drei Möglichkeiten: als Kopie, als Referenz oder als Pointer.

```
// Return as copy
Token next_token() {
    Token wuppie = Token("wuppie", 1, 4);   // will be deleted automatically
    Token bar = Token("bar", 7, 10);        // not used, will be deleted
        automatically

    return wuppie;
}
int main() {
    Token x = next_token();                 // copy constructor; no need to
        free
```

```
}
```

```
// Return as pointer
Token* next_token() {
    Token* foo = new Token("foo", 9, 35);   // will be free'd manually
    Token* bar = new Token("bar", 7, 10);   // leaves a memory hole!!!

    return foo;
}
int main() {
    Token* x = next_token();                // only the pointer (i.e. address) will be copied
    ...
    delete x;                               // caller needs to free this object
}
```

```
// Return as C++ reference
Token& next_token() {
    Token* foo = new Token("foo", 9, 35);   // will be free'd manually
    Token* bar = new Token("bar", 7, 10);   // leaves a memory hole!!!

    return *foo;
}
int main() {
    Token& x = next_token();                // no copy, `x` is just a new alias for `foo`
    ...
    delete &x;                              // caller needs to free this object
}
```

Die Rückgabe per Kopie (Standardfall in C/C++) würde ein lokales Objekt auf dem Stack (im Beispiel wäre das `wuppie`) als Kopie zurückgeben.

- Vorteil: Der Compiler kümmert sich um die Freigabe der lokalen Variable `wuppie`, d.h. nach Beendigung des Funktionsaufrufs wird das Objekt automatisch vom Stack entfernt. Da hierbei einfach der Stackpointer zurückgesetzt wird, ist diese "Freigabe" eine sehr preiswerte Operation.[1]
- Nachteil: Der Aufrufer darf nicht einfach auf das Objekt auf dem Stack zugreifen (dieses ist ja nach Beendigung der Funktion nicht mehr gültig). Deshalb muss das Objekt bei der Rückgabe kopiert werden (Copy-Konstruktor). Zusätzlich erfolgt beim Aufrufer oft noch eine Zuweisung, bei der die Attribute des Objekts vermutlich erneut kopiert werden. Dies kann (je nach Aufbau der Objekte) sehr teuer sein!

Die Rückgabe per Pointer erfordert ein Objekt, welches innerhalb der Funktion erzeugt wird und dessen Lebensdauer über das Funktionsende hinausreicht. Das Objekt muss in diesem Fall also auf dem Heap angelegt werden.

- Vorteil: Die Rückgabe erfordert lediglich die Kopie der Adresse des Objekts (also des Pointers). Hier handelt es sich vereinfacht betrachtet um einen Integer, d.h. diese Operation ist relativ preiswert.
- Nachteil: Das Objekt muss vom Aufrufer wieder freigegeben werden, sobald es nicht mehr benötigt wird. Dies muss man explizit programmieren!

[1] Anmerkung: Man spricht trotzdem von "Freigabe" des Objekts, obwohl lediglich der Stackpointer zurückgesetzt wird und das Objekt zunächst auf dem Stack noch vollständig ist. Es kann und wird aber im weiteren Verlauf des Programms überschrieben.

Die Rückgabe per C++-Referenz erfordert ebenfalls ein Objekt, welches innerhalb der Funktion erzeugt wird und dessen Lebensdauer über das Funktionsende hinausreicht. Das Objekt muss in diesem Fall also wieder auf dem Heap angelegt werden.

- Vorteil: Die Rückgabe erfordert keinerlei Kopien, da sich die Referenz `x` an das Objekt `foo` bindet und lediglich einen neuen Alias für dieses Objekt darstellt.
- Nachteil: Das Objekt muss vom Aufrufer wieder freigegeben werden, sobald es nicht mehr benötigt wird. Dies muss man explizit programmieren!

Es hat sich gezeigt, dass der Umgang mit den Heap-Ressourcen sehr fehleranfällig ist. Ein Aspekt dabei ist, dass man häufig die Freigabe der Objekte vergisst oder dass die Programmpfade so unübersichtlich sind, dass man nicht genau weiss, ob und wann man Objekte freigeben soll (denken Sie an Exceptions).

3.2 Smart-Pointer als Lösung

Während man in Sprachen wie Java die Speicherverwaltung komplett dem Compiler überlässt oder wie in Rust mit strikten Ownership-Modellen arbeitet, hat man in C++ die sogenannten Smart-Pointer erdacht. Diese ersetzen den direkten Umgang mit den einfachen Pointern (auch als *raw pointer* bezeichnet) und lösen das Problem der Freigabe der verwalteten Ressourcen.[2] Es gibt verschiedene Modelle, insbesondere gibt es die Variante *unique pointer*, bei der immer nur genau ein Smart-Pointer gleichzeitig eine bestimmte Ressource besitzen darf, und die *shared pointer*, bei der mehrere Smart-Pointer gleichzeitig die selbe Ressource verwalten können. Sobald die Lebensdauer des *unique pointer* oder des letzten *shared pointer* endet, wird die verwaltete Ressource automatisch vom Smart-Pointer freigegeben.

Das folgende Beispiel arbeitet mit einer selbst implementierten Variante der *shared pointers*. Dabei ist die Klasse `SmartToken` ein Smart-Pointer für Objekte vom Typ `Token`:

```
1  void fluppie() {
2      // new smart pointer for token "wuppie":
3      // wuppie lives on the stack, the token lives on the heap (`new`)
4      SmartToken wuppie = SmartToken(new Token("wuppie", 1, 4));
5
6      if (bla==42) {
7          // fluppie shares resource with wuppie
8          SmartToken fluppie = SmartToken(wuppie);
9          // fluppie2 shares resource with wuppie
10         SmartToken fluppie2(wuppie);
11
12         // now there are 3 smart pointers sharing the same resource (token "
               wuppie")
13
14         // new smart pointer for token "foo"
15         SmartToken foo = SmartToken(new Token("foo", 9, 35));
16     }   // fluppie, fluppie2, foo will be removed from the stack - foo releases
            its resource
17
18     // wuppie is the only smart pointer with shared resource "wuppie"
19
```

[2] Dereferenzierung von Null-Pointern oder nicht initialisierten Pointern, Nutzung von `delete` für Pointer, die nicht mit **`new`** erstellt wurden, mehrfaches `delete`, Speicherlöcher durch Vergessen von `delete`, Dangling Pointer, verwitwete Objekte, …

```
20 } // wuppie will be removed from the stack - wuppie releases its resource
```

Im Beispiel wird mit `new Token("wuppie", 1, 4)` ein neues Token-Objekt auf dem Heap angelegt. Der Smart-Pointer `wuppie` übernimmt die Ressource im Konstruktor und verwaltet den Pointer. Wichtig ist zu beobachten: Das Token wird auf dem Heap angelegt, während der Smart-Pointer `wuppie` eine normale lokale ("automatische") Variable ist und auf dem Stack liegt.

In der Kontrollstruktur werden weitere Smart-Pointer angelegt. Die ersten beiden (`fluppie`, `fluppie2`) teilen sich die Ressource (den Pointer auf das Token) mit `wuppie`. Es wird kein neues Token angelegt oder kopiert. Der dritte Smart-Pointer `foo` verwaltet ein weiteres Token.

Mit der Kontrollstruktur endet auch die Lebensdauer der lokalen Variablen `fluppie`, `fluppie2` und `foo`, sie werden automatisch vom Stack entfernt. Da `foo` der letzte Smart-Pointer ist, der das Token "foo" verwaltet, wird hier die Ressource freigegeben. Bei `fluppie` und `fluppie2` werden nur die Smart-Pointer auf dem Stack entfernt, die verwaltete Ressource (Token "wuppie") bleibt erhalten, da die noch von einem anderen Smart-Pointer verwaltet wird.

Mit dem Ende der Funktion endet auch die Lebensdauer des Smart-Pointers `wuppie`. Er wird automatisch vom Stack entfernt, und da er im Beispiel der letzte Smart-Pointer ist, der das Token "wuppie" verwaltet, wird dabei automatisch der Pointer zu "wuppie" wieder freigegeben.

Ein Smart-Pointer soll entsprechend folgende Eigenschaften haben:

- Verwendung soll analog zu normalen Pointern sein (Operatoren `*` und `->` überladen)
- Smart-Pointer haben niemals einen undefinierten Wert: entweder sie zeigen auf ein Objekt oder auf `nullptr`[3]
- Kopieren von (*shared*) Smart-Pointern führt dazu, dass sich mehrere Smart-Pointer das verwiesene Objekt *teilen*
- Smart-Pointer löschen sich selbst (und das verwiesene Objekt, falls kein anderer Smart-Pointer mehr darauf zeigt), wenn die Smart-Pointer ungültig werden (bei Verlassen des Scopes bzw. bei explizitem Aufruf von `delete` auf einen Pointer auf einen Smart-Pointer)
- Es gibt keine verwitweten Objekte mehr: Wenn mehrere Smart-Pointer auf das selbe Objekt zeigen, darf erst der letzte Smart-Pointer das Objekt aus dem Heap löschen
- Smart-Pointer funktionieren nur für mit **new** erzeugte Objekte

Weitere übliche Eigenschaften, die wir auf diesem Blatt aber vereinfachend ignorieren[4]:

- Smart-Pointer sollen für beliebige Klassen nutzbar sein (Template-Klasse)
- Dereferenzierung von nicht existierenden Objekten (d.h. der Smart-Pointer zeigt intern auf `nullptr`) führt nicht zum Programmabsturz, sondern zu einer Exception

3.3 Reference Counting

Smart-Pointer werden erzeugt, indem sie entweder einen Pointer auf die zu verwaltende Ressource bekommen (Konstruktor) oder eine Referenz auf ein anderes Smart-Pointer-Objekt (Copy-Konstruktor).

[3] Sie müssen für `nullptr` den g++ auf C++11 oder höher umstellen (`--std=c++11`) und den Header `<cstddef>` includen.
[4] Templates haben wir hier noch nicht behandelt, Exceptions werden wir gar nicht betrachten

Im Smart-Pointer wird entsprechend der Pointer auf die zu verwaltende Ressource gespeichert.

Für die Bestimmung, wie viele Smart-Pointer sich eine Ressource teilen, muss ein Zähler implementiert werden. Sobald sich ein weiterer Smart-Pointer die selbe Ressource teilt, muss dort auch der Zähler (per Pointer!) übernommen werden und entsprechend inkrementiert werden. Im Destruktor muss der Zähler dekrementiert werden. Falls dabei der Zähler den Wert 0 erreicht, werden die Pointer auf die Ressource und den Zähler freigegeben.

Bei einer Zuweisung verfährt man analog.

```
class SmartToken {
public:
    /**
     * Constructor
     *
     * Constructs a new smart pointer from a raw pointer, sets the reference
     * counter to 1.
     *
     * @param p is a raw pointer to the token to be shared
     */
    SmartToken(Token* p = nullptr);

    /**
     * Copy constructor
     *
     * Constructs a new smart pointer from another smart pointer, increments
     * the reference counter.
     *
     * @param sp is another smart pointer
     */
    SmartToken(const SmartToken& sp);

    /**
     * Destructor
     *
     * Decrements the reference counter. If it reaches zero, the shared token
     * will be free'd.
     */
    ~SmartToken();

    /**
     * Assignment
     *
     * Changes the shared token, thus we need first to perform something like
     * the destructor, followed by something like the constructor.
     *
     * @param sp is another smart pointer
     */
    SmartToken& operator=(const SmartToken& sp);

private:
    Token* pObj;        ///< Pointer to the current shared token
    RefCounter* rc;     ///< Pointer to the reference counter (used for the
        current token)
};

class RefCounter {
```

```
public:
    /**
     * Default constructor
     */
    RefCounter();

    /**
     * Increment count
     */
    void inc();

    /**
     * Decrement count
     */
    void dec();

    /**
     * Compare the counter with zero
     *
     * @return true if n==0, false otherwise
     */
    bool isZero() const;

    // Hide copy constructor and assignment operator
    RefCounter(const RefCounter&) = delete;
    RefCounter& operator=(const RefCounter&) = delete;

private:
    unsigned int n;     ///< How many SmartToken share ownership of "our"
        object?
};
```

3.4 Dereferenzierung von Smart-Pointern

(*Anmerkung*: Dies ist ein Vorgriff auf die Lektion “Operatoren”. Betrachten und implementieren Sie die vorgegebenen Operatoren einfach wie normale Methoden.)

Pointer lassen sich dereferenzieren, d.h. man greift direkt auf das verwiesene Objekt zu. Dies lässt sich auch für Smart-Pointer erreichen, indem die beiden Dereferenzierungsoperatoren überladen werden.

```
class SmartToken {
public:
    /**
     * Dereferences the smart pointer
     *
     * @return a reference to the shared token
     */
    Token& operator*();

    /**
     * Dereferences the smart pointer
     *
     * @return a pointer to the shared token
     */
```

```
    Token* operator->();
};
```

Damit lässt sich das folgende Verhalten realisieren (Vergleich *raw* Pointer vs. Smart-Pointer):

```
Token* foo = new Token("foo", 9, 35);                        // raw pointer foo
SmartToken wuppie = SmartToken(new Token("wuppie", 1, 4));  // smart pointer
    wuppie

// Access via token pointer
cout << (*foo).lexem    << endl;    // "foo"
cout << foo->lexem      << endl;    // "foo"

// Access via smart pointer
cout << (*wuppie).lexem << endl;    // "wuppie"
cout << wuppie->lexem   << endl;    // "wuppie"
```

Dabei ist die Form "`->`" eine vereinfachte Darstellung von `(*ptr).`, d.h. ein Pointer (linke Seite des Ausdrucks) wird dereferenziert und man greift auf Attribute oder Methoden des verwiesenen Objekts zu (rechte Seite des Ausdrucks).

4 Aufgaben

4.1 A7.1: Klasse für Token (1P)

Implementieren Sie in C++ die Klasse `Token` mit der folgenden Schnittstelle:

```
class Token {
public:
    /**
     * Constructs a new token object.
     *
     * @param l is a pointer to the text of the token (to be copied)
     * @param r is the row in input where this token was found
     * @param c is the column in input where this token starts
     */
    Token(const char* l, int r, int c);

    /**
     * Destructs the token object and free's the stored lexem.
     */
    ~Token();

private:
    char* lexem;    ///< Pointer to the text of the token
    int row;        ///< Row in input where this token was found
    int col;        ///< Column in input where this token starts
};
```

Trennen Sie Deklaration und Implementierung.

Der Konstruktor muss den übergebenen `char*` kopieren, d.h. Sie müssen die Länge des übergebenen C-Strings

bestimmen, ausreichend viel Speicher mit `new` für `char* lexem` reservieren und danach den String kopieren (C-Funktion).

Sorgen Sie dafür, dass der Speicher beim Vernichten eines `Token`-Objekts wieder korrekt freigegeben wird.

Bei Bedarf können Sie zusätzliche Attribute und Methoden hinzufügen.

Testen Sie Ihre `Token`-Klasse an selbst gewählten Beispielen.

4.2 A7.2: Implementierung eines einfachen Tokenizers (1P)

Erstellen Sie eine Funktion `void tokenize(const string& input, vector<Token>& tokens)`, die einen gegebenen String als Eingabe erhält und diesen in Tokens (Wörter) splittet. Nutzen Sie Referenzen, um die Token-Liste zu aktualisieren. Testen Sie die Funktion mit verschiedenen Eingabestrings und geben Sie die Tokens aus.

4.3 A7.3: Reference Counter (1P)

Implementieren Sie nun die Klasse `RefCounter` mit der obigen Schnittstelle. Auch hier können Sie bei Bedarf zusätzliche Attribute und Methoden hinzufügen.

Testen Sie Ihre `RefCounter`-Klasse an selbst gewählten Beispielen.

4.4 A7.4: Smart-Pointer (3P)

Implementieren Sie nun die Smart-Pointer für `Token`-Objekte mit folgender Signatur (wie oben, leicht erweitert):

```
class SmartToken {
public:
    /**
     * Constructor
     *
     * Constructs a new smart pointer from a raw pointer, sets the reference
     * counter to 1.
     *
     * @param p is a raw pointer to the token to be shared
     */
    SmartToken(Token* p = nullptr);

    /**
     * Copy constructor
     *
     * Constructs a new smart pointer from another smart pointer, increments
     * the reference counter.
     *
     * @param sp is another smart pointer
     */
    SmartToken(const SmartToken& sp);

```

```
    /**
     * Destructor
     *
     * Decrements the reference counter. If it reaches zero, the shared token
     * will be free'd.
     */
    ~SmartToken();

    /**
     * Assignment
     *
     * Changes the shared token, thus we need first to perform something like
     * the destructor, followed by something like the constructor.
     *
     * @param sp is another smart pointer
     */
    SmartToken& operator=(const SmartToken& sp);

    /**
     * Dereferences the smart pointer
     *
     * @return a reference to the shared token
     */
    Token& operator*();

    /**
     * Dereferences the smart pointer
     *
     * @return a pointer to the shared token
     */
    Token* operator->();

    /**
     * Comparison
     *
     * @param sp is another smart pointer
     * @return true, if `sp` shares the same token
     */
    bool operator==(const SmartToken& sp) const;

private:
    Token* pObj;        ///< Pointer to the current shared token
    RefCounter* rc;     ///< Pointer to the reference counter (used for the
        current token)
};
```

Bei Bedarf können Sie zusätzliche Attribute und Methoden hinzufügen.

Testen Sie Ihre `SmartToken`-Klasse an selbst gewählten Beispielen sowie an den obigen Beispielen.

4.5 A7.5: Ringpuffer (4P)

Ein Ringpuffer ist eine Form der abstrakten Datenstruktur “Warteschlange” (*Queue*), die nur eine beschränkte Anzahl n von Elementen (Datensätzen) aufnehmen kann. Die Daten werden nach dem FIFO-Prinzip über die

Funktion `writeBuffer()` am Ende der Schlange eingefügt und mit der Funktion `readBuffer()` vom Anfang der Schlange entnommen.

Aus Effizienzgründen wird bei `readBuffer()` nur das erste Element zurückgeliefert, das gelesene Element wird aber (noch) nicht aus dem Ringpuffer entfernt. Über ein Attribut `head` merkt man sich stattdessen, wo das nächste zu lesende Element liegt (auf dem Platz hinter dem aktuell gelesenen). Ist der Puffer voll, wird bei `writeBuffer()` das älteste Element entfernt und das neue Element auf dem frei gewordenen Platz im internen Array `elems` eingefügt.

Unser Ringpuffer ist auf Elemente vom Typ `SmartToken` festgelegt. Es wird davon ausgegangen, dass diese Elemente Smart-Pointer mit der *shared pointer*-Semantik sind.[5] Da die `SmartToken` selbst (zum Teil) eine Pointersemantik implementiert haben (man kann die Smart-Pointer dereferenzieren), vermeiden wir Pointer auf die Smart-Pointer in der Schnittstelle und arbeiten stattdessen mit C++-Referenzen bzw. Kopien: Bei `writeBuffer()` wird ein `SmartToken` per konstanter C++-Referenz übergeben, und bei `readBuffer()` wird eine Kopie des gelesenen `SmartToken` zurückgeliefert.

Der Puffer kann effizient durch ein zur Laufzeit angelegtes **Array** mit `size` (Template-Parameter) Plätzen zur Speicherung der Pointer auf die Elemente realisiert werden. Die Ringstruktur wird durch Modulo-Operationen auf den Array-Indizes realisiert.

Implementieren Sie nun den Ringpuffer für `SmartToken`-Objekte mit folgender Signatur:

```
class RingBuffer {
public:
    /**
     * Constructor that creates a new ring buffer for max. `size` elements
     *
     * Initialises the attributes and allocates memory for `size` elements
     * of type `SmartToken` and let the pointer `elems` point to this new
     * array
     *
     * @param size is the max. number of elements that can be stored
     */
    RingBuffer(unsigned int size);

    /**
     * Destructor
     *
     * free's the dynamically allocated array `elems`
     */
    ~RingBuffer();

    /**
     * Reading the first (oldest) element
     *
     * If an element has been read, the `head` points to the next element
     * and `count` is decremented. The read element is **not** released.
     *
     * @return Returns (a copy of) the first (i.e. oldest) element of the
     * buffer, but does not (yet) release it; returns an empty `SmartToken`
     * if buffer is empty
     */
```

[5] Wenn Sie die obigen Aufgaben richtig gelöst haben, haben Sie genau diese Semantik vorliegen.

```
31      SmartToken readBuffer();
32
33      /**
34       * Adding a new element
35       *
36       * Appends the new element to the end of the queue. If the buffer is
37       * full, the oldest element will be overwritten by the new element. The
38       * old element will take care of releasing its memory (smart pointer).
39       *
40       * @param data is a reference to the element to be added
41       */
42      void writeBuffer(const SmartToken& data);
43
44  private:
45      unsigned int count;     ///< number of elements currently stored in the
            buffer
46      unsigned int head;      ///< points to the beginning of the buffer (oldest
            element)
47      unsigned int size;      ///< length of array `elems`
48      SmartToken* elems;      ///< array with `size` places of type `SmartToken`,
             dynamically allocated
49  };
```

Bei Bedarf können Sie zusätzliche Attribute und Methoden hinzufügen.

Testen Sie Ihre `RingBuffer`-Klasse an selbst gewählten Beispielen. Überzeugen Sie sich insbesondere vom korrekten Zugriff auf die Ringstruktur und prüfen Sie, ob die Smart-Pointer wie gewünscht arbeiten. Prüfen Sie hierzu auch die `RefCounter` der beteiligten Smart-Pointer. Welche Sonderfälle können Sie identifizieren?

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 7b260e5 (homework: finalize B07 (#379), 2025-10-03)